Artificial Intelligence Coursework
London Taxi Search Agent
Module Code: CMP-N206-0

Student Name – Farhan Ahmed Sahol
Student ID – A00027052

1. Introduction

Artificial Intelligence (AI) enables systems to operate autonomously by perceiving their environment, making decisions, and taking actions to achieve defined objectives. One of the key areas within AI is pathfinding, where an intelligent agent must determine the most efficient route in a given environment.

This coursework focuses on the design and implementation of a taxi agent operating within a grid-based environment inspired by London. The environment is represented as a structured grid where each cell corresponds to a possible road position. The taxi agent is required to navigate from its starting position, locate a passenger, and transport them to a specified destination.

To simulate real-world challenges, the environment includes constraints such as blocked roads and traffic zones. These elements introduce complexity by forcing the agent to consider both path length and movement cost. As a result, the agent must make intelligent decisions rather than simply selecting the shortest route.

Several search algorithms are implemented and evaluated in this project, including Breadth-First Search (BFS), Depth-First Search (DFS), Greedy Best-First Search, and A* Search. Each algorithm follows a different strategy for exploring the environment and selecting paths.

The objective of this coursework is to analyse the performance of these algorithms and determine which approach is most effective for efficient navigation in a constrained environment.

2. Problem Formulation

The taxi navigation problem can be modelled using an agent-based framework, where an intelligent agent interacts with an environment to achieve a goal.

The main objectives of the taxi agent are:

- Navigate to the passenger location
- Pick up the passenger
- Deliver the passenger to the destination

- Minimise movement cost and total steps

This problem can also be interpreted as a graph search problem, where:

- Each grid cell represents a node
- Movements between cells represent edges
- Costs are associated with movement

The challenge is to determine which search strategy allows the agent to efficiently explore the environment and identify the best possible route.

2.1 Agent–Environment Model

The taxi system can be described using an agent–environment interaction model.

Agent:
The taxi that operates within the grid.

Environment:

- Grid-based road network
- Blocked positions (obstacles)
- Traffic zones with higher movement cost
- Passenger and destination locations

Percepts:

- Current position
- Valid neighbouring cells
- Locations of obstacles and traffic
- Passenger and destination

Actions:

- Move up
- Move down
- Move left
- Move right

The agent selects actions based on its current state and goal.

2.2 PEAS Framework

The taxi agent can be defined using the PEAS framework:

Performance Measure:

- Successful pickup and drop-off
- Minimum steps taken
- Reduced total cost
- Avoidance of unnecessary movement

Environment:

- Grid-based road system
- Blocked cells
- Traffic zones

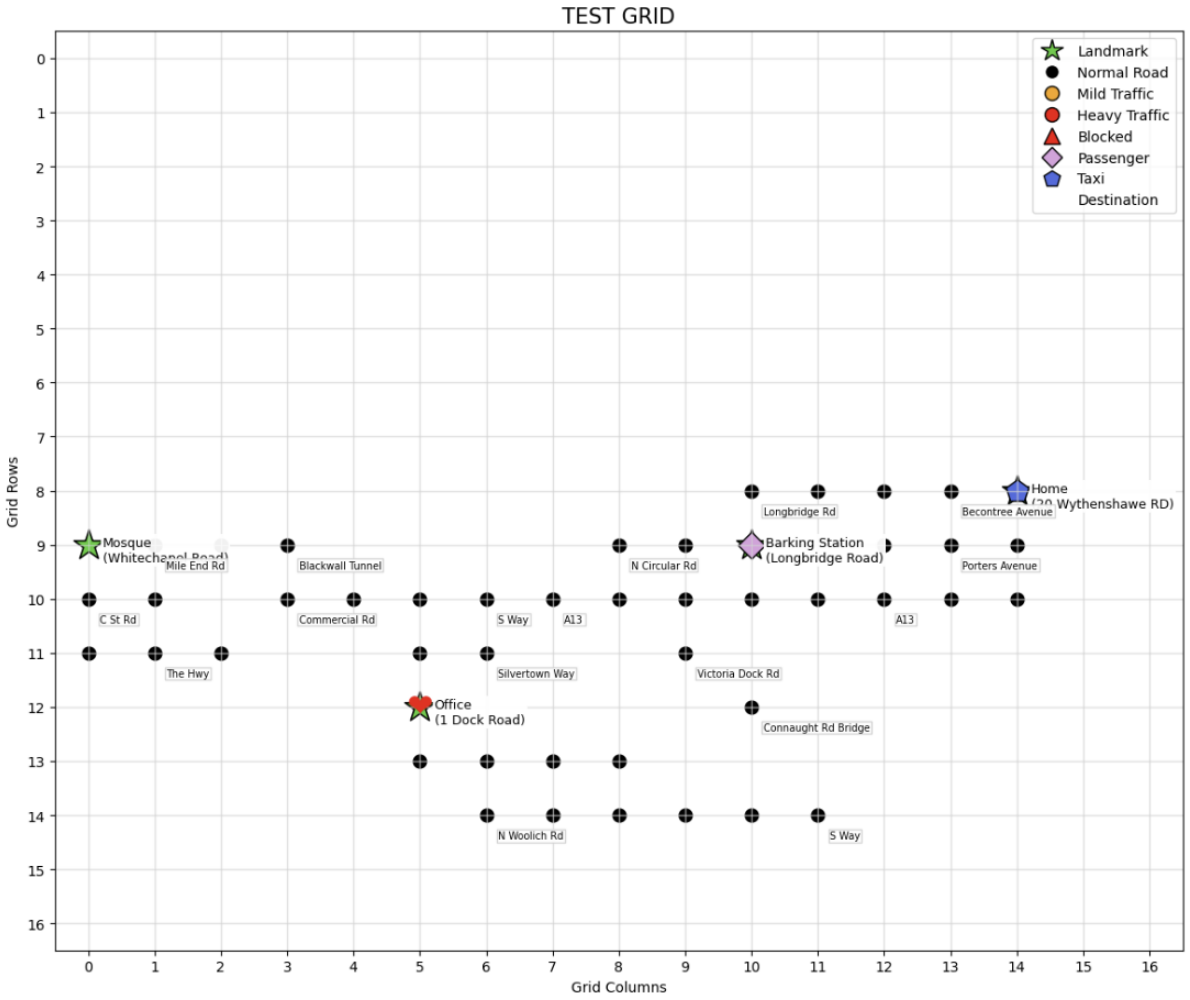
Actuators:

- Movement in four directions

Sensors:

- Position tracking
- Detection of valid and invalid cells
- Awareness of passenger and destination

3. Environment Design



The environment is implemented as a grid-based model where each cell represents a potential position for the taxi. This approach simplifies the problem while still capturing key elements of real-world navigation.

3.1 Grid World

The grid contains several types of cells:

- **Normal cells:** Standard movement cost
- **Blocked cells:** Cannot be entered
- **Traffic cells:** Higher movement cost
- **Special cells:** Passenger and destination points

The taxi can only move between adjacent cells in four directions.

3.2 Constraints and Rules

The environment operates under the following rules:

- The taxi cannot move outside the grid boundaries
- Blocked cells act as obstacles
- Traffic zones increase movement cost
- The agent must complete both pickup and drop-off

These constraints make the problem more complex and require intelligent decision-making.

4. Taxi Agent Design

The taxi agent is designed as a goal-based agent that selects actions based on its objective of reaching a target location.

4.1 Percepts

The agent receives:

- Its current position
- Passenger location
- Destination location
- Available valid moves

This information allows the agent to evaluate possible paths.

4.2 Actions

The agent performs movement actions:

- Up
- Down
- Left
- Right

Each action moves the taxi to a neighbouring cell.

5. Performance Measure

The performance of the taxi agent is evaluated using a reward and penalty system:

- Pickup reward: +5

- Drop-off reward: +20
- Movement cost: -1 per step
- Failure penalty: -20

This structure encourages efficient navigation and discourages unnecessary movement.

Additional evaluation metrics include:

- Path length
- Total cost
- Nodes explored
- Computational efficiency

6. Decision-Making Strategy

Different search algorithms are used to determine the taxi's route.

6.1 Breadth-First Search (BFS)

BFS explores nodes level by level and guarantees the shortest path in terms of steps when all movements have equal cost. However, it does not consider traffic costs, making it less effective in weighted environments.

6.2 Depth-First Search (DFS)

DFS explores one path deeply before backtracking. It requires less memory but does not guarantee the shortest path and may result in inefficient routes.

6.3 Greedy Best-First Search

Greedy search selects nodes based only on heuristic distance to the goal. It is fast but may produce suboptimal paths because it ignores actual movement cost.

6.4 A Search*

A* combines actual cost and heuristic distance to find the most efficient path. It balances exploration and efficiency, making it the most suitable algorithm for environments with traffic and obstacles.

7. Experimental Evaluation

Several scenarios are used to evaluate algorithm performance, including short routes, longer paths, and environments with obstacles and traffic.

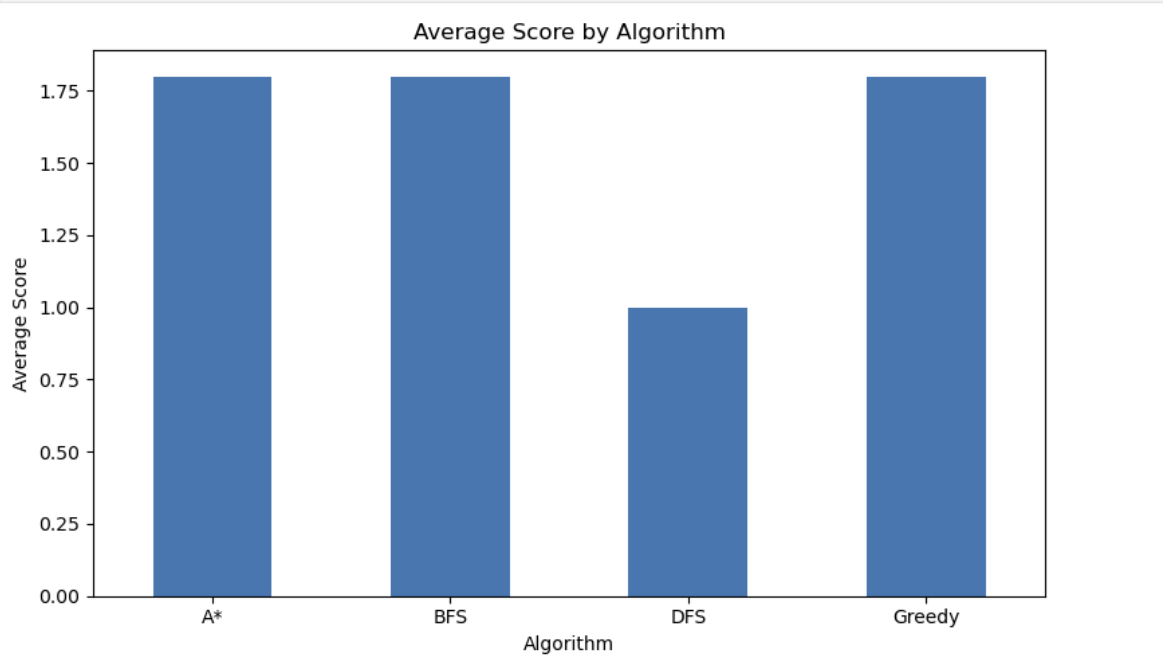
	Scenario	Algorithm	Success	Steps	Score	Nodes Explored	Path Cost	Path Length
0	Scenario 1 - Home to Office	A*	True	13	12	24	17.0	14
1	Scenario 1 - Home to Office	BFS	True	13	12	26	22.0	14
2	Scenario 1 - Home to Office	DFS	True	17	8	21	26.0	18
3	Scenario 1 - Home to Office	Greedy	True	13	12	15	17.0	14
4	Scenario 2 - Office to Mosque	A*	True	8	17	10	8.0	9
5	Scenario 2 - Office to Mosque	BFS	True	8	17	22	8.0	9
6	Scenario 2 - Office to Mosque	DFS	True	8	17	15	8.0	9
7	Scenario 2 - Office to Mosque	Greedy	True	8	17	10	8.0	9
8	Scenario 3 - Mosque to Barking	A*	False	0	-20	28	inf	0
9	Scenario 3 - Mosque to Barking	BFS	False	0	-20	28	inf	0
10	Scenario 3 - Mosque to Barking	DFS	False	0	-20	28	inf	0
11	Scenario 3 - Mosque to Barking	Greedy	False	0	-20	28	inf	0
12	Scenario 4 - Barking to Home	A*	True	5	20	7	5.0	6
13	Scenario 4 - Barking to Home	BFS	True	5	20	18	5.0	6
14	Scenario 4 - Barking to Home	DFS	True	5	20	13	5.0	6
15	Scenario 4 - Barking to Home	Greedy	True	5	20	7	5.0	6
16	Scenario 5 - Home to Mosque via Barking Pickup	A*	False	0	-20	24	inf	0
17	Scenario 5 - Home to Mosque via Barking Pickup	BFS	False	0	-20	30	inf	0
18	Scenario 5 - Home to Mosque via Barking Pickup	DFS	False	0	-20	28	inf	0
19	Scenario 5 - Home to Mosque via Barking Pickup	Greedy	False	0	-20	24	inf	0

The results show that:

- BFS finds shortest paths but explores many nodes
- DFS produces longer and less efficient paths
- Greedy search is fast but not always optimal
- A* provides the best balance between accuracy and efficiency

In addition to these results, it is important to consider how each algorithm behaves under different conditions. In more complex environments with multiple obstacles, BFS and DFS tend to explore a larger number of nodes, increasing computational cost. This makes them less efficient for large-scale problems.

In contrast, A* performs more effectively by using heuristic guidance to focus on promising paths. This reduces unnecessary exploration and improves overall efficiency. Greedy search, while fast, may struggle when the heuristic does not accurately represent the true cost of reaching the goal.



These findings highlight the importance of selecting an appropriate algorithm depending on the structure and constraints of the environment.

8. Explainability

Explainability ensures that the decisions made by the agent can be understood.

In this system, the agent's behaviour can be explained by:

- The algorithm used
- The path selected
- Cost calculations

For example:

- BFS selects paths based on shortest steps
- Greedy focuses on closeness to the goal
- A* balances cost and distance

This makes the system transparent and interpretable.

9. Critical Reflection

Although the system performs effectively, there are several limitations:

- Static environment
- Simplified traffic modelling

- Limited real-world complexity

Future improvements could include:

- Dynamic traffic conditions
- Real-world map integration
- Multiple agents
- Machine learning techniques

10. Conclusion

This project demonstrates how search algorithms can be applied to a navigation problem. By comparing BFS, DFS, Greedy, and A*, algorithm choice significantly affects performance.

A* proves to be the most effective due to its ability to balance cost and heuristic guidance. The project highlights the importance of selecting appropriate algorithms for efficient decision-making in complex environments.

11. References

1. Russell, S. and Norvig, P. (2020) *Artificial Intelligence: A Modern Approach*. 4th edn. Pearson.

Norvig, P. (n.d.) *Search Algorithms*. Stanford University. Available at: https://stanford.edu/~norvig .

2. Dechter, R. and Pearl, J. (1985) 'Generalized Best-First Search Strategies and the Optimality of A*', *Journal of the ACM*, 32(3), pp. 505–536.

3. LaValle, S. M. (2006) *Planning Algorithms*. Cambridge: Cambridge University Press.

4. GeeksforGeeks (2024) *Search Algorithms in AI*. Available at: https://www.geeksforgeeks.org (Accessed: 19 March 2026).